

Vortrag — Handpapier

Entwurf und Evaluation eines automatisierten Migrationswerkzeugs von Katalon-Groovy-Tests zu Selenium-Pytest

Konstantin Lucius Regenhardt · HTW Berlin · Prüfer: Prof. Dr. Jochen Wittmann · Zweitgutachter: Ankit Kumar · 15 Minuten

Roter Faden — im Einstieg setzen, am Ende aufgreifen:
Katalon macht den Einstieg leicht und den Ausstieg teuer — aber genau die starre, proprietäre Struktur, die den Lock-in erzeugt, ist berechenbar genug, um sie automatisiert aufzulösen.

Zahlen-Spickzettel

Testskripte übersetzt	7 von 10 = 70 %, 3 offen (30 %)
Object Repository	10 von 10 <code>.rs</code> → JSON
Weitere Artefakte	1 Variablendatei, 2 Runtime-Dateien, 4 Konfigurationsdateien, 1 Datendatei
<code>pytest --collect-only</code>	7 collected, 0 Importfehler, 0.46 s
Testlauf	Katalon 10/10 grün · Python 7/7 grün
Boilerplate	18 → 7 Importzeilen
Laufzeit Migration	< 1 Sekunde vs. geschätzt hoher einstelliger bis zweistelliger Stundenbereich manuell
Setup	<code>sample-website-katalon-tests</code> , Windows 11, Python 3.13.0, pytest 8.4.1
Module im Migrator	7 Pipeline-Module, orchestriert von <code>main.py</code>
Katalon-Formate	<code>.tc</code> · <code>.rs</code> · <code>.glbl</code> · <code>.dat</code> — vier Endungen, alles XML

Ablauf mit laufender Uhr

Taktmarken fett markiert — wenn du dort bist, liegst du richtig.

Uhr	Abschnitt	Dauer	PDF-Sprungmarke
0:00	1 Einstieg	0:30	Titelseite
0:30	2 Motivation & Problemstellung	1:45	Kap. 1.1 / 1.2
2:15	3 Ziel & vier Leitfragen	0:45	Kap. 1.3
3:00	4 Was hinter einem Katalon-Projekt liegt	2:15	Kap. 3.2.1 (Ordnerbaum), 3.1.2, 3.1.3
5:15	5 Zielstruktur	0:45	Kap. 3.3.1 (Ordnerbaum Ziel)
6:00	6 Migrationspipeline	1:15	Kap. 4.2 (Übersichtsdiagramm)
7:15	7 Transpilation & Regex	1:30	Kap. 4.3 (Aktivitätsdiagramm), 5.3.1
8:45	8 Vorher / Nachher	1:15	Kap. 5.3.1.1 + 5.3.1.2
10:00	9 Evaluation	2:15	Kap. 6.2, 6.3, 6.4, 6.5
12:15	10 Grenzen	0:45	Kap. 4.5 / 5.4
13:00	11 Fazit & Ausblick	1:00	Kap. 7
14:00	Ende — 1 Min. Puffer		

1 — Einstieg · 0:00–0:30 · Titelseite

Kernbotschaft: „Ich habe ein Werkzeug gebaut, das ein komplettes Katalon-Testprojekt in ein lauffähiges Python-Projekt mit Selenium und Pytest überführt — und gemessen, wie weit das automatisch trägt.“

Direkt in die Ausgangslage. Keine Agenda-Folie vorlesen, das kostet bei 15 Minuten nur Zeit.

2 — Motivation & Problemstellung · 0:30–2:15 · Kap. 1.1 / 1.2

Kernbotschaft: „Das Problem war nicht, dass Katalon schlecht war. Das Problem war, dass wir nicht mehr rauskamen.“

- Team baut mit Katalon Studio Testinfrastruktur für interne Geschäftssoftware auf, sie wächst über den Großteil der Software.
- Bruchstelle: fortgeschrittene Anforderungen — automatisiertes Berechnen von Datumseinträgen, Debugging über die CLI, tiefere Kontrolle über die Testausführung — hängen an teureren Lizenzstufen. Preise stehen in keinem Verhältnis zur Leistung.
- Das ist Vendor Lock-in im Lehrbuchsinn (Shapiro/Varian): proprietäre Formate, plattformeigene Skriptsprache, Funktionen hinter Lizenzstufen.
- Neuschreiben war keine Option — zu viel Investition steckt in den bestehenden Tests. Also Migration als Proof of Concept.
- Entscheidender Punkt: Migration ist **nicht** Groovy → Python. Ein Katalon-Projekt ist Testskript **plus** Object Repository, Global Variables in Profilen, testeigene Variablen, Testdaten, Custom Keywords — alles proprietär und über Pfadkonventionen verdrahtet.

Wichtigste Minute des Vortrags. Hier entsteht die Fallhöhe für alles Weitere.

3 — Ziel & vier Leitfragen · 2:15–3:00 · Kap. 1.3

Kernbotschaft: „Entwurf, Implementierung und Evaluation eines Migrators — mit vier messbaren Fragen.“

- Wie viel des Quellprojekts wird automatisch übersetzt?
- Wie hoch ist der tatsächliche Automatisierungsgrad?
- Ist das Ergebnis funktional äquivalent zum Ausgangsprojekt?
- Welche Fälle bleiben offen und erfordern manuelle Nacharbeit?

Grundlagen (Kap. 2) in max. zwei Sätzen streifen oder weglassen — die Prüfer kennen sie.

4 — Was hinter einem Katalon-Projekt liegt · 3:00–5:15 · Kap. 3.2.1, 3.1.2, 3.1.3

Kernbotschaft: „In der IDE sieht man einen Test. Im Dateisystem sind es zwei Dateien in zwei Ordnern, verbunden nur über eine Pfadkonvention.“

Stärkster Teil — hier zeigst du etwas, das nicht offensichtlich ist. Zeit lassen.

- Ein „Test Case“ = `.tc`-Datei (umbenanntes XML: Metadaten, GUID, Werte der testeigenen Variablen) **plus** `Scripts/<Pfad>/<Testname>/Script<Zufallszahl>.groovy` mit der eigentlichen Logik.
- Die Zufallsnummer hat keinen Bezug zur `.tc`-Datei, die GUID taucht im Skript nie auf — die einzige Verbindung ist der gespiegelte Pfad. → **Ordnerbaum in Kap. 3.2.1 zeigen**.
- Variablen werden im Skript „roh“ verwendet: nicht deklariert, nicht initialisiert, kein Typ.
- Object Repository (Kap. 3.1.2): `.rs` = XML mit `WebElementEntity`, mehrere Selektor-Strategien als Key-Value-Paare plus `selectorMethod`, das die bevorzugte bestimmt.
- Profile (Kap. 3.1.3): `.glibl` = XML mit `GlobalVariableEntity`. Nebenbefund: als „protected“ markierte Werte sind nur in der UI maskiert — in der Datei stehen sie im Klartext.
- Data Files: `.dat` = XML-Metadaten, die nur auf die eigentliche Datei zeigen.
- Vier Endungen, ein Format: XML. Die Endungen verschleiern den Inhalt.

Die Wendung, die den Rest trägt: Genau diese Starrheit ist der Hebel. Weil die IDE die Struktur vollständig vorgibt, ist sie vorhersagbar — und damit automatisiert auflösbar.

5 — Zielstruktur · 5:15–6:00 · Kap. 3.3.1

Kernbotschaft: „Pytest schreibt fast nichts vor — deshalb spiegle ich bewusst die Katalon-Struktur, damit die Tests ihre Pfade behalten.“

- `src/`-Layout trennt gespiegelte Katalon-Strukturen von der Projektwurzel.
- `tests/` spiegelt die Test-Case-Ordner · `variables/` testeigene Variablen als Python-Klassen · `object_repository/` Test Objects als JSON · `profiles/global_variables.py` · `data/` Testdaten.
- Dazu Laufzeithelfer und Konfiguration, damit das Projekt ohne Vorarbeit startet.

Zügig — das ist Kontext, kein Ergebnis.

6 — Migrationspipeline · 6:00–7:15 · Kap. 4.2 (Übersichtsdiagramm)

Kernbotschaft: „Vier Phasen: Struktur anlegen, Tests transpilieren, Kontext übernehmen, Projekt lauffähig machen.“

- **Initialisierung:** Quell- und Zielpfad, rekursiver Scan, Ordnerstruktur im Ziel spiegeln — die unveränderten Pfade sind integral.
- **Transpilation** der Test Cases (Details nächster Abschnitt).
- **Übernahme der Kontextstrukturen:** `.rs` → JSON · Global Variables → Python-Klasse · testeigene Variablen → Python-Klasse pro Test · Testdaten unverändert kopiert.
- **Abschluss:** `pytest.ini`, `requirements.txt`, `.gitignore`, `README.md`, Laufzeithelfer.
- Sieben spezialisierte Module, von `main.py` orchestriert. Modulnamen nicht einzeln vorlesen — Zahl und Prinzip genügen.
- **Robustheitsentscheidung:** nicht übersetzbare Skripte werden nicht verworfen, sondern nach `src/unreadable_tests/` gelegt. Der Durchlauf bleibt stabil, die Problemfälle bleiben sichtbar.

7 — Transpilation & Regex · 7:15–8:45 · Kap. 4.3 (Aktivitätsdiagramm), 5.3.1

Kernbotschaft: „Sieben Schritte, die den Groovy-Test schrittweise auf seine Logik reduzieren und dann in Python wieder aufbauen.“

Einlesen → Kommentare und Imports entfernen → Testzeilen extrahieren → parsen (Klasse, Methode, Parameter) → transformieren → Python-Code erzeugen → zum vollständigen Test zusammensetzen.

Zu Regex nicht die Sprache erklären, sondern das Prinzip: die Muster arbeiten nicht unabhängig, sondern als aufeinander aufbauende Kette —

- `katalon_lines_pattern`: Zeilenblock → Matches mit Gruppen für Klasse, Methode, Parameter
- `fto_as_param_pat`: erkennt `findTestObject()` als ersten Parameter, wenn ein Komma folgt
- `fto_param_str_pattern`: extrahiert das String-Argument daraus

Entwickelt und visualisiert mit Regex101, anschließend in Python gegengeprüft, damit die Ausdrücke zur tatsächlichen Laufzeitumgebung passen. Vollständige Übersicht: **Anhang 8.1**.

8 — Vorher / Nachher · 8:45–10:00 · Kap. 5.3.1.1 + 5.3.1.2

Kernbotschaft: „Der Trick ist die Zweiteilung: Format-Konvertierung zur Build-Time, Auflösung zur Laufzeit.“

Konzeptionell interessantester Teil — hier ruhig langsam sprechen.

- **Build-Time:** `.rs` → JSON, Struktur bleibt identisch, nur das Format ändert sich.
- **Laufzeit:** `find_katalon_test_object()` liest `selectorMethod`, sucht den passenden Wert aus der `selectorCollection` und lokalisiert über `By.CSS_SELECTOR` bzw. `By.XPATH` (bei `BASIC` und allem anderen).
- **Ergebnis:** Der generierte Test kennt die Strategie gar nicht, er übergibt nur einen Pfad — strukturell identisch zum Katalon-Original.

```
Groovy
WebUI.click(findTestObject('All_Users/view_all_users_btn'))

Python
kh.find_katalon_test_object(self.driver, 'All_Users/view_all_users_btn').click()
```

Drei Transformationsarten am Beispiel (Kap. 5.3.1.2):

- Assertions auf Seiteninhalte: `WebUI.verifyTextPresent(...)` → `assert '...' in self.driver.page_source`
- Objektzugriffe über den Laufzeithelfer: `setText(...)` → `.clear()` + `.send_keys(variables.david)`
- Variablenzugriffe auf die neuen Strukturen: `GlobalVariable.user4name` → `GlobalVariable.USER4NAME`

Pfade und Objektbezeichner werden nicht verworfen, sondern in ein neues Zugriffsschema übertragen — deshalb bleibt die Bindung zwischen Testlogik und Kontext nach der Migration erhalten.

9 — Evaluation · 10:00–12:15 · Kap. 6.2 → 6.3 → 6.4 → 6.5

Kernbotschaft: „7 von 10 Testskripten automatisch, alle umgebenden Strukturen vollständig — und das Ergebnis läuft.“

Setup kurz vorweg: `sample-website-katalon-tests`, Windows 11, Python 3.13.0, pytest 8.4.1.

- **Kap. 6.2 Migrationsergebnis** (Tabelle + Konsolenausschnitt): 7/10 Testskripte = **70 %**. Object Repository **10/10** nach JSON. Dazu 1 Variablendatei, 2 Runtime-Dateien, 4 Konfigurationsdateien, 1 Datendatei.
- **Kap. 6.3 Strukturelle Integrität:** `pytest --collect-only` importiert alle generierten Dateien fehlerfrei und findet genau die 7 erwarteten Tests. Testlauf: Katalon 10/10 grün, Python 7/7 grün — die Differenz sind exakt die drei nicht übersetzten Fälle, **keine zusätzlichen Fehler durch die Migration**.
- **Kap. 6.4 Codequalität:** Boilerplate von 18 auf 7 Importzeilen, Testlogik in Standard-Python mit Pytest-Konventionen, keine proprietären Keywords mehr.
- **Kap. 6.5 Aufwand:** Migration unter einer Sekunde gegenüber geschätzt hohem einstelligen bis zweistelligen Stundenbereich manuell. Wichtiger als die Einzelzahl: **die Skalierung**. Der Migrator wächst linear mit der Testanzahl, der manuelle Aufwand überproportional, weil mit jedem Test die Querbezüge zu Variablen, Objekten und Daten dazukommen.

Die 30 % selbst ansprechen, bevor jemand fragt — das wirkt souveräner als eine Verteidigung auf Nachfrage.

Kernbotschaft: „Bewusst auf die häufigsten Migrationspfade fokussiert, nicht auf Vollständigkeit.“

- Custom Keywords (eigene Klassen in `Keywords/`, Weiterleitungen über `Libs/CustomKeywords.groovy`).
- Stark verschachtelte Testlogik und Parameterlisten, von der Norm abweichende Groovy-Konstrukte, Schleifen und Kontrollstrukturen.
- Kommentare werden derzeit vollständig entfernt.
- Methodisch ehrlich: ein Beispielprojekt mit 10 Tests — die 70 % sind ein belastbarer Indikator, keine allgemeingültige Quote.

Kernbotschaft: „Kein vollständiger Ersatz für manuelle Migration, aber ein wirksamer Ausstiegspfad.“

- 70 % der Testskripte und alle umgebenden Strukturen automatisch, Projekt nach minimalem Setup lauffähig.
- Der Ausstieg aus einem proprietären Testökosystem ist automatisierbar — nicht restlos, aber weit genug, dass er wirtschaftlich wird.
- Drei Richtungen: (1) mehr Katalon-Keyword-Methoden abdecken, (2) Parsing-Regeln für Kommentare, tiefere Verschachtelung, eigene Variablen/Methoden/Klassen und Schleifen erweitern, (3) Fehlerklassifikation und -ausgabe verbessern, damit die manuelle Nacharbeit gezielter wird.
- Einstiegssatz wieder aufgreifen.

Wenn die Zeit knapp wird

In dieser Reihenfolge kürzen:

- Abschnitt 5 (Zielstruktur) auf zwei Sätze
- Abschnitt 10 (Grenzen) ins Fazit integrieren
- Abschnitt 7 (Regex) auf das Prinzip ohne die drei Muster
- Aufwandsvergleich in Abschnitt 9 auf einen Satz

Nicht kürzen: Abschnitt 4 und Abschnitt 8. Das sind die beiden Stellen, an denen du etwas zeigst, das nicht jeder schon weiß.

Wahrscheinliche Rückfragen

- „**Warum Regex und kein echter Parser / AST?**“ → Prototyp im Rahmen einer konkreten betrieblichen Aufgabe; die relevanten Katalon-Zeilen folgen einem stark eingeschränkten Muster (`WebUI.`-Aufrufe). Ein Groovy-AST wäre der robustere Weg für Schleifen und Verschachtelung — steht genau so im Ausblick.
- „**Woran genau sind die 3 Tests gescheitert?**“ → Konstrukte außerhalb des abgedeckten Musterraums; sie landen in `src/unreadable_tests/` statt verloren zu gehen. Eine Fehlerkategorisierung ist bewusst als nächster Schritt benannt.
- „**Ist funktionale Äquivalenz wirklich nachgewiesen?**“ → Geprüft wurden strukturelle Integrität (`collect-only`) und erfolgreiche Ausführung beider Projekte. Ein systematischer Verhaltensvergleich Schritt für Schritt wäre der nächste Ausbau — das offen so sagen.
- „**Lässt sich das auf andere Katalon-Projekte übertragen?**“ → Ja, soweit sie die Standard-Katalon-Strukturen nutzen. Projektindividuelle Custom Keywords sind der limitierende Faktor.
- „**Warum nicht in Katalon bleiben und nur die Lizenz kaufen?**“ → Kosten standen in keinem Verhältnis, und das Problem wiederholt sich bei der nächsten Funktion. Die Migration löst die Abhängigkeit, statt sie zu verschieben.

Kapitelübersicht für schnelles Springen im PDF

Kap.	Titel
1.1 / 1.2 / 1.3	Motivation · Problemstellung · Ziel & Aufbau
2.1–2.4	Automatisiertes Testen · Low-Code · Vendor Lock-in · Regex
3.1.1–3.1.5	Katalon: Test Cases · Object Repository · Profile · Testdaten · Custom Keywords
3.2.1–3.2.5	Verschachtelungen (Ordnerbäume)
3.3.1–3.3.4	Python-Zielstruktur · Object Repository · Testdaten · Konfiguration
4.1–4.5	Hintergrund · Architekturüberblick · Transpilation · Kontextstrukturen · Limitierungen
5.1–5.4	Techstack · Projektstruktur · Umsetzung (5.3.1 Transpilation) · Grenzen
6.1–6.5	Kriterien · Migrationsergebnis · Strukturelle Integrität · Codequalität · Aufwand
7	Fazit und Ausblick
8.1 / 8.2	Anhang: Regex-Muster · Katalon Projektstruktur